

Modul 03

NLP Fundamentals

Natural Language Processing dengan Python



Act 1

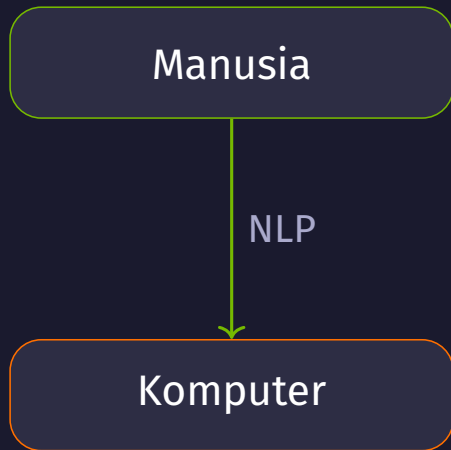
Apa itu NLP?

Bagaimana komputer memahami bahasa manusia?

Definisi NLP

- ▶ **Natural Language Processing**
- ▶ Cabang AI yang membuat komputer **memahami** & **memproses** bahasa manusia
- ▶ Bukan sekadar manipulasi string
- ✓ Bahasa: Inggris, Indonesia, Mandarin, dll
- ✓ Bentuk: teks, suara

Contoh: Google Translate, Siri, ChatGPT, mesin pencari



Aplikasi NLP di Dunia Nyata

- ✓ **Mesin pencari** (Google, Bing) → memahami maksud query
- ✓ **Penerjemahan mesin** (Google Translate) → lintas bahasa
- ✓ **Analisis sentimen** → ulasan produk, pemantauan merek
- ✓ **Chatbot & asisten** (Siri, Alexa) → percakapan
- ✓ **Klasifikasi teks** → penyaring spam, kategorisasi berita
- ✓ **Peringkasan** → ringkasan otomatis dokumen



Tantangan Bahasa Manusia

Bahasa Inggris:

- ▶ Morfologi sederhana
run → runs → running
- ▶ Urutan kata tetap (SVO)
- ▶ Imbuhan terbatas (-s, -ed, -ing)

Bahasa Indonesia:

- ▶ Imbuhan kaya
jalan → menjalani → dijalankan
- ▶ Imbuhan: me-, -kan, -i, pe-, -an, ber-
- ▶ Kata serapan dan singkatan

Bahasa berbeda butuh library berbeda: `nltk` + `spaCy` untuk Inggris, `nlp-id` untuk Indonesia

Satu kata dasar, belasan bentuk permukaan

Intuisi



Mesin melihat tujuh kata berbeda. Manusia melihat satu gagasan. Lemmatisasi adalah usaha menutup jarak itu.

Peta Perjalanan Modul 03



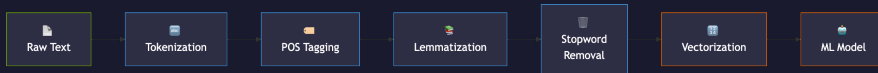
- ▶ Notebook 01 menempuh keempat bagian di CPU
- ▶ Notebook 02 mengulang alur yang sama di GPU dengan NVIDIA RAPIDS
- ▶ Bagian D adalah jembatan langsung ke Modul 04 (LLM) dan Modul 05 (RAG)

Act 2

Membersihkan Teks

Kenapa langkah paling membosankan justru paling menentukan

Pipeline NLP: Dari Teks ke Model



Tahapan utama: dari teks mentah sampai siap masuk model ML

Tokenisasi: Memecah Teks

"Saya belajar NLP di Navasena."



Saya

belajar

NLP

di

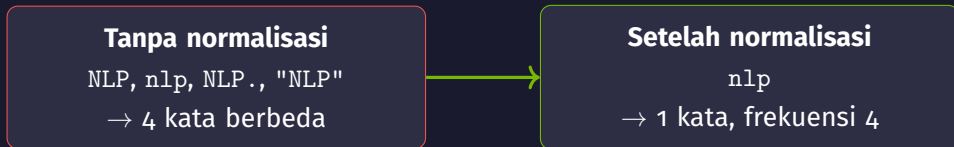
Navasena

.

- ▶ **Token** = unit terkecil yang diproses model
- ▶ Di Bagian A satu token = satu kata
- ▶ Di Bagian D satu token = potongan subword
- ✓ NLTK: `word_tokenize()`
- ✓ nlp-id: `Tokenizer().tokenize()`

Kenapa huruf kecil dan tanda baca dibuang duluan

Intuisi



Setiap variasi ejaan yang tidak disatukan menjadi satu dimensi fitur tambahan yang harus dipelajari model dari nol.

Stopword: Kata yang Tidak Membedakan

- ▶ Kata yang muncul di hampir semua dokumen
- ▶ Indonesia: yang, di, dari, dan, adalah
- ▶ Inggris: the, is, are, of
- ▶ Membuangnya memperkecil dimensi fitur
- ✓ `nlp-id: StopWord().get_stopword()`
- ✓ `NLTK: stopwords.words('english')`

`["saya", "belajar", "NLP", "di", "Navasena"]`



buang stopwords



`["belajar", "NLP", "Navasena"]`

Jebakan: Stopword bukan daftar universal. Untuk analisis negasi, membuang tidak justru menghapus informasi yang paling menentukan.

Lemmatisasi vs Stemming

Stemming

- ▶ Memotong imbuhan secara mekanis
- ▶ Cepat, tidak butuh kamus
- ▶ Hasilnya boleh bukan kata: mempelajari → pelajar

Lemmatisasi

- ▶ Berbasis kamus dan aturan morfologi
- ▶ Lebih lambat, lebih akurat
- ▶ Hasilnya selalu kata: mempelajari → ajar

nlp-id menyediakan `Lemmatizer().lemmatize()`. **Sastrawi** masih sering dikutip untuk stemming Indonesia, tetapi rilis terakhirnya tahun 2016 — perlakukan sebagai referensi historis.

POS Tagging: Menandai Kelas Kata

"Saya membaca buku di perpustakaan."



Catatan: label di slide memakai tagset UD (ilustratif); `nlp-id` memakai NNP/VB/NN/IN.

- ▶ **POS** = *part of speech*, kelas kata
- ▶ Dipakai untuk menyaring kandidat topik (ambil kata benda saja)
- ▶ Juga untuk penerjemahan mesin dan ekstraksi informasi

Tokenisasi Frasa: Entitas Multi-kata

"Universitas Indonesia adalah kampus terbaik di Jakarta."

Tokenisasi kata

```
["universitas",  
"indonesia", ...]
```

nama lembaga terpecah dua

Tokenisasi frasa

```
["universitas  
indonesia", ...]
```

satu nama, satu token

- ▶ `PhraseTokenizer` mengelompokkan frasa nomina jadi satu token
- ▶ Berguna sebelum analisis frekuensi: "kecerdasan buatan" tidak tercerai
- ▶ Bukan NER — ia menemukan frasa, tetapi tidak memberi label jenis entitas

Ringkasan Act 2

Langkah	Yang dilakukan
Normalisasi	Huruf kecil, buang angka dan tanda baca, rapikan spasi
Tokenisasi	Pecah teks jadi kata (atau frasa)
Stopword	Buang kata yang tidak membedakan dokumen
Lemmatisasi	Kembalikan kata ke bentuk dasar berbasis kamus
POS tagging	Tandai kelas kata untuk penyaringan lanjutan

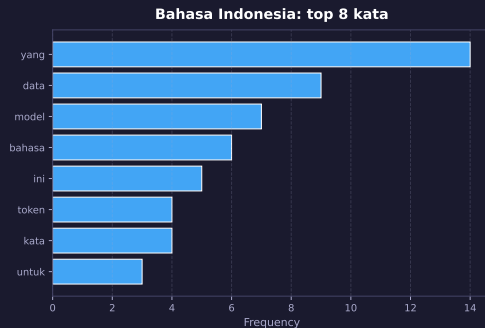
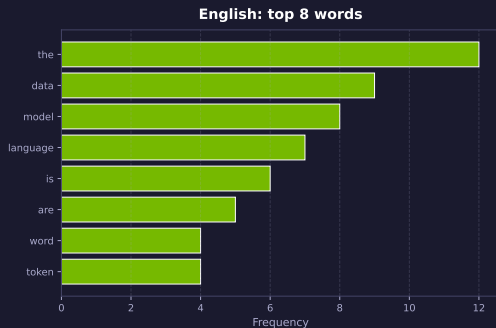
Keluaran Act 2: daftar token bersih — bahan baku untuk semua act berikutnya.

Act 3

Menganalisis Teks

Apa yang dibicarakan dokumen ini, dan siapa yang disebut?

Frekuensi Kata: Menebak Topik



Stopword Inggris: the, is, are · Stopword Indonesia: yang, di, dari, dan

Frekuensi tanpa pembersihan tidak memberi tahu apa pun Intuisi

Teks mentah, top-5

1. yang
2. di
3. dan
4. adalah
5. untuk

Daftar ini sama untuk dokumen apa pun.

Setelah dibersihkan, top-5

1. nlp
2. bahasa
3. model
4. analisis
5. sentimen

Baru di sini topiknya terbaca.

Pembersihan bukan pekerjaan rumah tangga — ia yang menentukan apakah hasil analisis punya isi.

Word Cloud: Cantik, tapi Hati-hati

Kelebihan

- ▶ Sekali lihat langsung menangkap tema
- ▶ Bagus untuk presentasi dan laporan awam

Kekurangan

- ▶ Mata sulit membandingkan luas huruf
- ▶ Tata letaknya acak, bukan informasi
- ▶ Tidak bisa dibaca sebagai angka

Aturan praktis

Word cloud untuk
menarik perhatian.

Bar chart untuk
mengambil keputusan.

Named Entity Recognition (NER)

"Budi Santoso bekerja di Navasena di Jakarta sejak 2024."

Budi Santoso

PERSON

bekerja

VERB

di

Navasena

ORG

di

Jakarta

LOC

✓ **PERSON** (orang) · **ORG** (organisasi) · **LOC** (lokasi) · **DATE** (tanggal)

spaCy melabeli kota/negara sebagai GPE (bukan LOC).

NER untuk Apa? Tiga Kasus Nyata

Kontrak

Tarik nama pihak, nilai, dan tanggal berlaku dari ratusan dokumen PDF.

Berita

Pantau setiap penyebutan nama perusahaan di media, lalu gabungkan dengan sentimennya.

Layanan

Isi otomatis formulir tiket dari keluhan pelanggan berbentuk teks bebas.

Pola yang sama: teks bebas masuk, data terstruktur keluar.

NER Bahasa Indonesia: Tiga Jalur

Jalur	Kelebihan	Batasan
spaCy xx_ent_wiki_sm	Multibahasa, ringan	Akurasi sedang
Model NER IndoBERT di Hugging Face	Akurasi terbaik untuk Indonesia	Unduhan besar, butuh GPU agar cepat
PhraseTokenizer (nlp-id)	Tanpa model, sangat cepat	Menemukan frasa, tidak memberi label

spaCy belum menyediakan pipeline khusus bahasa Indonesia.

Ringkasan Act 3

- ✓ **Frekuensi kata** menebak topik — selalu setelah pembersihan
- ✓ **Word cloud** untuk menarik perhatian, **bar chart** untuk keputusan
- ✓ **NER** mengubah teks bebas menjadi data terstruktur
- ✓ Untuk Indonesia, NER berkualitas berarti memakai model transformer

Act 3 masih menggambarkan teks. Act 4 mulai menilai-nya.

Act 4

Menilai Teks

Sentimen dan klasifikasi — dan satu kesalahan yang sering lolos

Analisis Sentimen: Dua Generasi

Generasi 1 — leksikon

- ▶ Kamus kata beserta skor sentimennya
- ▶ Skor kata dijumlahkan
- ▶ Cepat, transparan, tanpa GPU
- ▶ Contoh: `TextBlob`

Generasi 2 — transformer

- ▶ Model dilatih dari teks nyata berlabel
- ▶ Membaca susunan kalimat, bukan menjumlah
- ▶ Butuh unduhan ratusan MB
- ▶ Contoh: `IndoBERT`

TextBlob: $\text{polarity} \in [-1,0; 1,0]$ · $\text{subjectivity} \in [0,0; 1,0]$

Skala Polaritas



Jebakan: Skor 0,0 punya dua arti yang sangat berbeda: kalimat benar-benar netral, atau tidak ada satu pun katanya yang ada di kamus. Leksikon tidak membedakan keduanya.

Batas Leksikon: Negasi

“the material is **not bad** at all”

not

bad

at all

leksikon menjumlahkan skor per kata

maksud manusia: pujian $\neq$ **hasil penjumlahan**

- ▶ Leksikon tidak membaca urutan, jadi negasi berlapis dan sarkasme mudah meleset
- ▶ Transformer membaca kalimat sebagai satu kesatuan — di situlah bedanya

"Pelayanan restoran itu lambat dan mengecewakan"



TextBlob: polarity = 0,00

Jebakan: Nol di sini bukan "netral". Kamus TextBlob hanya berbahasa Inggris, jadi tidak ada satu pun kata Indonesia yang dikenali. Nol berarti "tidak tahu", dan model tidak akan memberi tahu kamu bedanya.

Sentimen Indonesia dengan IndoBERT

```
1 sentimen = pipeline("sentiment-analysis",  
2     model="w11wo/indonesian-roberta-base-sentiment-classifier")  
3 sentimen("Gw suka bgt sama teknologi NLP nih")  
4 # [{'label': 'positive', 'score': 0.99}]
```

- ▶ Transformer yang di-*fine-tune* untuk bahasa Indonesia — tahan bahasa gaul
- ▶ Pola transfer learning yang sama dengan ResNet di Modul 02
- ▶ Mengenali tiga kelas: positif, negatif, dan netral
- ▶ Ekosistem: IndoBERT, IndoNLU, IndoLEM, NusaBERT (Hugging Face Hub)

Kenapa transformer tahan bahasa gaul

Intuisi

Leksikon

- ▶ Kamusnya disusun manusia dari teks baku
- ▶ gw, bgt, anjir tidak ada di dalamnya
- ▶ Menambah kata baru berarti menyunting kamus

Transformer

- ▶ Dilatih dari teks nyata: Twitter, ulasan, forum
- ▶ Bahasa gaul ikut terlatih karena memang ada di data
- ▶ Subword membuat kata tak dikenal tetap terwakili

Bedanya bukan kecerdasan model, melainkan dari mana pengetahuannya berasal: kamus yang disusun, atau data yang dikumpulkan.

Leksikon atau Transformer?

Cocok kalau

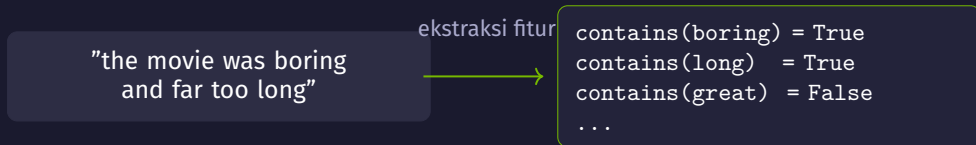
- ▶ Jutaan dokumen, butuh penyaringan kasar
- ▶ Tanpa GPU, harus jalan di CPU biasa
- ▶ Skor harus bisa dijelaskan per kata
- ▶ Teks Inggris baku

Kurang cocok kalau

- ▶ Teks Indonesia atau bahasa informal
- ▶ Negasi dan sarkasme penting
- ▶ Butuh kelas netral yang benar
- ▶ Akurasi lebih penting daripada kecepatan

Keduanya sering dipakai bersama: leksikon menyaring, transformer memutuskan.

Klasifikasi Teks: Bag-of-Words



- ▶ Urutan kata dibuang total — yang dihitung hanya kehadiran
- ▶ Setiap dokumen jadi vektor `True/False` sepanjang kosakata fitur
- ▶ Sederhana, tetapi inilah bentuk paling awal dari "teks sebagai vektor"

Naive Bayes: Kenapa Masih Diajarkan

- ▶ Menghitung $P(\text{kelas} \mid \text{fitur})$ lewat Teorema Bayes
- ▶ "Naive" karena menganggap setiap kata muncul saling bebas — asumsi yang salah, tetapi bekerja baik untuk teks
- ▶ Latihnya sangat cepat, bahkan di data besar
- ▶ Keputusannya **bisa dibaca manusia**

Fitur paling informatif

outstanding pos : neg = 13,0 : 1,0

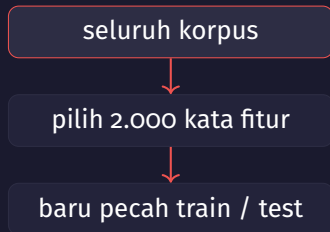
insulting neg : pos = 11,0 : 1,0

ludicrous neg : pos = 9,7 : 1,0

Dibaca: kata itu 13x lebih sering muncul di ulasan positif.

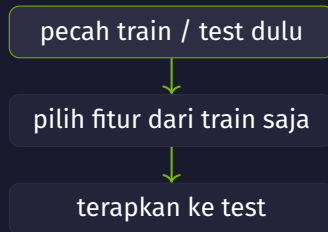
Kebocoran data: kesalahan yang tidak menimbulkan error Intuisi

Salah



data test ikut menentukan fitur
akurasi jadi terlalu bagus

Benar



test benar-benar belum pernah dilihat
akurasi bisa dipercaya

Aturan yang Sama Berlaku untuk Semuanya

Langkah	Yang dipelajari dari data
Seleksi fitur	Kata mana yang dipakai sebagai kolom
TfidfVectorizer	Kosakata dan bobot IDF
StandardScaler	Rata-rata dan simpangan baku
Imputasi nilai kosong	Median atau modus pengisi

Semuanya di-fit pada train, lalu diterapkan ke test. Melanggarnya tidak memunculkan error — hanya angka yang menyenangkan dan salah.

Baseline: Angka Pembanding yang Wajib

Akurasi model

82%

Baseline kelas mayoritas

50%

- ▶ **Baseline kelas mayoritas** = akurasi kalau selalu menebak kelas terbanyak
- ▶ Di `movie_reviews` kedua kelas seimbang, jadi baseline $\approx 50\%$
- ▶ Kalau 95% data satu kelas, akurasi 94% berarti model lebih buruk daripada menebak

Jebakan: Akurasi tanpa baseline adalah angka tanpa arti. Selalu laporkan keduanya.

Ringkasan Act 4

- ✓ Sentimen leksikon cepat dan transparan, tetapi buta terhadap negasi dan bahasa non-Inggris
- ✓ Transformer Indonesia (IndoBERT) tahan bahasa gaul dan mengenali kelas netral
- ✓ Bag-of-words membuang urutan kata — "not good" dan "good" bersinyal sama
- ✓ Apa pun yang belajar dari data harus di-fit pada train saja
- ✓ Akurasi selalu dilaporkan bersama baseline kelas mayoritas

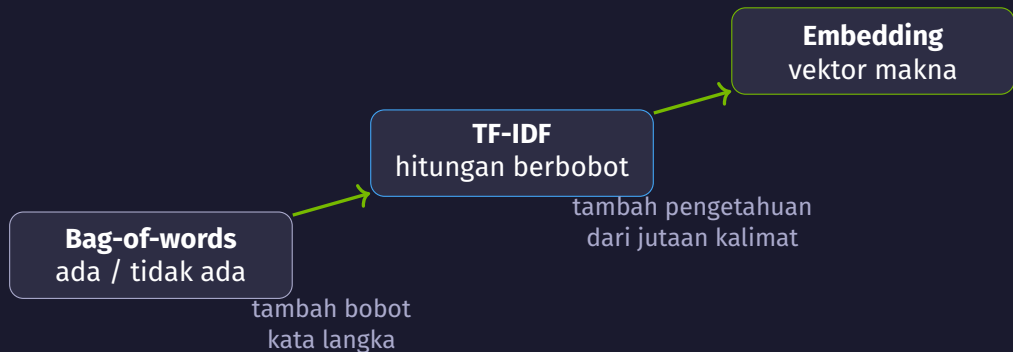
Masalah yang tersisa: hitungan kata tidak tahu bahwa "suka" dan "gemar" itu sama. Act 5 menyelesaikannya.

Act 5

Dari Kata ke Makna

Jembatan menuju LLM (Modul 04) dan RAG (Modul 05)

Tiga Anak Tangga Representasi Teks



Ketiganya mengubah teks jadi angka. Yang membedakan: berapa banyak *makna* yang ikut terbawa.

TF-IDF: Kata Langka Lebih Berharga

- ▶ **TF** (*term frequency*) — seberapa sering kata muncul di dokumen ini
- ▶ **IDF** (*inverse document frequency*) — seberapa jarang kata itu di seluruh koleksi
- ▶ Bobot tinggi = sering di sini, jarang di tempat lain

yang ada di 100% dokumen → bobot ≈ 0
model ada di 30% dokumen → bobot sedang
minhash ada di 2% dokumen → bobot tinggi

IDF melakukan otomatis apa yang daftar stopwords lakukan manual — menekan kata yang ada di mana-mana.

TF-IDF Buta terhadap Sinonim

K1: "Saya **suka makan** nasi goreng"

TF-IDF
similarity $\approx 0,32$

K2: "Aku **gemar menyan-
tap** nasi goreng"

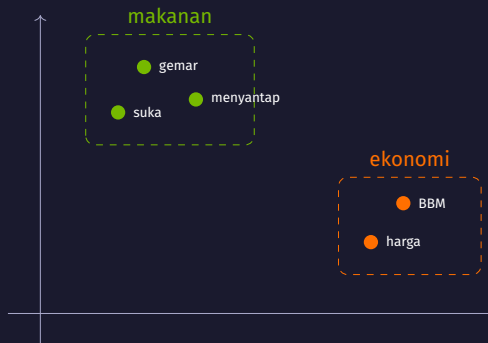
Embedding
similarity $\approx 0,85$

- ▶ Bagi TF-IDF, **suka** dan **gemar** adalah dua dimensi yang tidak berhubungan
- ▶ Yang tersisa hanya **nasi** dan **goreng** — makanya skornya rendah

Angka ilustratif; nilai persisnya dihitung di notebook 01 Section 9.

Embedding: makna sebagai koordinat

Intuisi



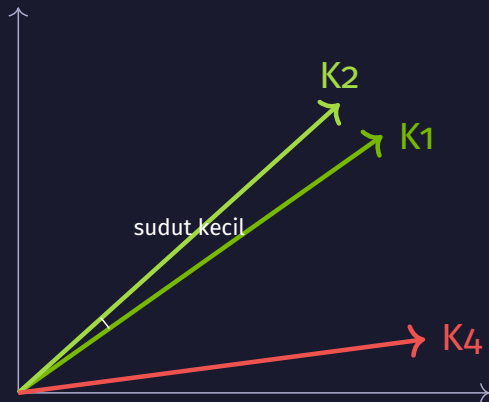
Kata bermakna mirip ditempatkan berdekatan, sehingga kemiripan makna bisa dihitung sebagai jarak.

Satu Kalimat, 384 Angka



- ▶ Panjang kalimat berapa pun selalu menghasilkan vektor sepanjang yang sama
- ▶ Model `paraphrase-multilingual-MiniLM-L12-v2` melayani puluhan bahasa sekaligus
- ▶ Karena multibahasa, kalimat Indonesia dan terjemahan Inggrisnya berdekatan

Cosine Similarity: Mengukur Kedekatan Arah

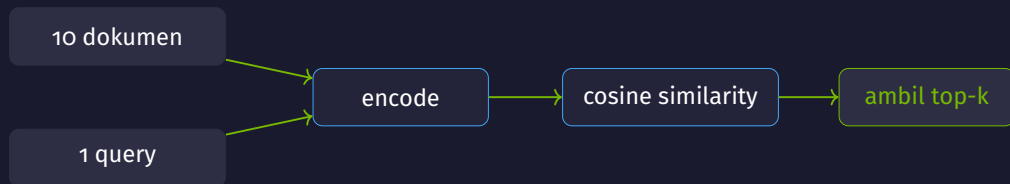


- ▶ Mengukur **sudut** antar vektor, bukan panjangnya
- ▶ Nilai 1 = arah sama persis, 0 = tegak lurus (tidak berhubungan)
- ▶ Panjang diabaikan, jadi dokumen panjang dan pendek tetap adil dibandingkan

Di kode

```
cosine_similarity(emb_query,  
emb_dokumen)
```

Semantic Search dalam Sepuluh Baris



Query *"puncak paling tinggi di Indonesia"* menemukan *"Gunung Kerinci adalah gunung berapi tertinggi di Indonesia"* — **tanpa satu pun kata yang sama**. TF-IDF tidak akan pernah menemukannya.

Inilah Inti RAG

Hari ini (Modul 03)

encode dokumen
encode query
cosine similarity
ambil top-k



Modul 05 (RAG)

encode dokumen ke **FAISS**
encode query
pencarian indeks
top-k → **LLM menyusun jawaban**

- ▶ Polanya identik — yang bertambah hanya indeks agar cepat untuk jutaan dokumen, dan LLM di ujung untuk menyusun kalimat jawaban
- ▶ Kamu sudah membangun bagian intinya hari ini

Tokenisasi Subword: Token ala LLM

```
1 tok = AutoTokenizer.from_pretrained("gpt2")
2 tok.tokenize("mempelajari")
3 # ['mem', 'pel', 'aj', 'ari'] -- potongan subword
```

- ▶ Potongan lebih kecil dari kata, lebih besar dari huruf
- ▶ Kosakata terbatas (~30–50 ribu subword) bisa menuliskan kata apa pun, termasuk yang belum pernah dilihat saat training
- ▶ Untuk bahasa Indonesia yang kaya imbuhan, *mempelajari* dan *dipelajari* berbagi potongan yang sama

Token adalah satuan biaya

Intuisi

kalimat yang maknanya sama, tokenizer GPT-2

Inggris



9 token

Indonesia



17 token

- ▶ Tokenizer GPT-2 dilatih dominan pada teks Inggris, jadi kata Indonesia terpecah lebih halus
- ▶ API LLM menagih **per token** — kalimat Indonesia jadi lebih mahal untuk isi yang sama
- ▶ Tokenizer multibahasa atau khusus Indonesia biasanya menekan rasio ini

Angka ilustratif; hitung sendiri di notebook 01 Section 10.

"Token" Mulai Modul 04

Konteks	Satu token =	Dipakai di
Bagian A modul ini	satu kata	<code>tokenizer.tokenize()</code>
Bagian D modul ini	satu subword	<code>AutoTokenizer</code>
Modul 04 dan seterusnya	satu subword	<code>max_new_tokens</code> , batas konteks, tagihan API

Mulai besok, kata "token" selalu berarti subword.

Ringkasan Act 5

- ✓ TF-IDF membobot kata langka lebih tinggi, tetapi tetap mencocokkan ejaan
- ✓ Embedding memetakan makna jadi koordinat — sinonim berdekatan
- ✓ Cosine similarity mengukur sudut, sehingga panjang dokumen tidak mengganggu
- ✓ Semantic search = encode, bandingkan, ambil top-k — inti RAG
- ✓ Tokenisasi subword menentukan biaya sebuah bahasa di API LLM

Semua ini berjalan di CPU. Act 6 menjalankan alur yang sama di GPU.

Act 6

NLP di GPU

NVIDIA RAPIDS — alur yang sama, skala yang berbeda

Kenapa NLP Klasik Butuh GPU

- ▶ Menyiapkan korpus LLM berarti memproses **miliaran** dokumen
- ▶ Operasi teks bersifat seragam dan berulang — persis yang disukai ribuan core GPU
- ▶ Preprocessing yang butuh berjam-jam di CPU bisa selesai dalam hitungan menit

CPU

belasan core
satu string cepat

GPU

ribuan core
ribuan string sekaligus

Praktik: notebook 02_nlp_on_steroids

Stack RAPIDS untuk NLP

Library	Padanan CPU	Peran
cuDF	pandas	DataFrame di memori GPU
nvtext	—	Operasi teks GPU: <code>tokenize</code> , <code>ngrams_tokenize</code> , <code>minhash</code>
cuML	scikit-learn	TF-IDF, Logistic Regression, clustering

- ▶ Sudah terpasang di runtime GPU Colab — tinggal `import`
- ▶ `nvtext` tidak punya padanan di `pandas`: operasinya memang lahir di GPU

Nol Perubahan Kode: cudf.pandas & cuml.accel

```
1 # CPU: pandas + sklearn biasa
2 python pipeline.py
3
4 # GPU: file yang SAMA, nol perubahan kode
5 python -m cudf.pandas -m cuml.accel pipeline.py
```

- ▶ `cudf.pandas` mengambil alih `import pandas` dan mengalihkan operasinya ke GPU
- ▶ `cuml.accel` melakukan hal yang sama untuk estimator scikit-learn
- ▶ Operasi yang belum didukung GPU otomatis *fallback* ke CPU — kode tidak crash
- ▶ Akurasi identik; yang berubah hanya **waktu**

GPU tidak selalu menang

Intuisi



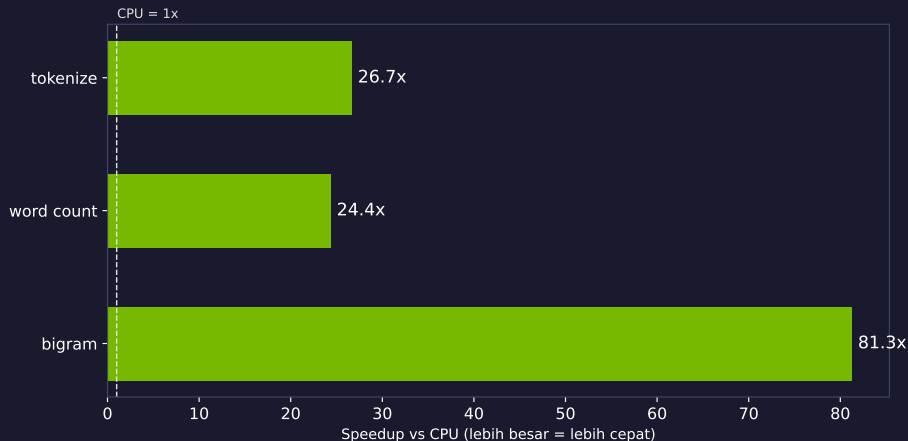
- ▶ GPU membayar biaya tetap: inisialisasi CUDA dan pemindahan data CPU↔GPU
- ▶ Untuk data kecil, biaya tetap itu lebih mahal daripada komputasinya sendiri
- ▶ **Aturan emas benchmark:** jangan pernah mengukur pemanggilan GPU pertama — lakukan *warm-up* dulu

nvtext: Operasi yang Tidak Ada di pandas

Operasi	Gunanya
<code>str.token_count()</code>	Hitung token per baris tanpa membentuk daftar
<code>str.tokenize()</code>	Pecah seluruh kolom jadi token sekaligus
<code>str.ngrams_tokenize(2)</code>	Bangun bigram langsung di GPU
<code>str.minhash()</code>	Sidik jari dokumen untuk deteksi duplikat

Makin intensif operasi string-nya, makin lebar kemenangan GPU — karena ribuan core mengerjakan ribuan baris serentak.

Seberapa Cepat? (Colab T4, 150 ribu paragraf Wikipedia ID)



Diukur langsung di notebook 02_nlp_on_steroids — angka di mesinmu bisa berbeda.

MinHash: Dari Miliaran Perbandingan Jadi Satu groupby

Cara lugu

bandingkan semua pasangan
150.000 dokumen
 $\approx$ 11 miliar perbandingan



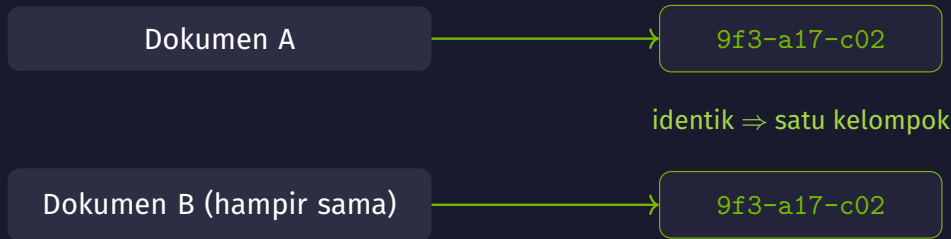
MinHash

tiap dokumen $\rightarrow$ sidik jari
sidik jari sama = mirip
 $\approx$ satu groupby

- ▶ Duplikat di korpus training berarti model menghafal dan komputasi terbuang
- ▶ Notebook memakai satu sidik jari utuh, jadi hanya duplikat nyaris identik yang tertangkap
- ▶ Sistem produksi (MinHash-LSH) memakai banyak *band* agar kemiripan parsial ikut terdeteksi

Kenapa sidik jari bisa menggantikan perbandingan

Intuisi



Dokumen mirip menghasilkan sidik jari sama dengan probabilitas tinggi. Masalah "cari yang mirip" berubah jadi "kelompokkan yang sama" — dan yang kedua jauh lebih murah.

Dari Lab ke Produksi: NeMo Curator

Hari ini (1 T4, 150 ribu paragraf)	Produksi (ratusan GPU, miliaran dokumen)
<code>tokenize, normalize</code> di <code>nvtext</code>	Identifikasi bahasa, pembersihan, penyaringan kualitas
<code>MinHash + groupby</code>	Fuzzy dedup <code>MinHash-LSH</code> terdistribusi
<code>TF-IDF + Logistic Regression</code>	Classifier kualitas dokumen

Bedanya cuma skala. Langkah-langkahnya sama persis dengan yang kamu jalankan hari ini.

Ringkasan Act 6

- ✓ `cudf.pandas` dan `cuml.accel` memindahkan kode lama ke GPU tanpa mengubahnya
- ✓ `nvtext` menyediakan operasi teks yang tidak ada padanannya di `pandas`
- ✓ GPU menang di skala; untuk data kecil ia justru bisa lebih lambat
- ✓ Warm-up wajib sebelum mengukur apa pun di GPU
- ✓ MinHash mengubah masalah kuadratik jadi satu `groupby`

Peta library NVIDIA: RAPIDS hari ini, NeMo dan TensorRT-LLM di Modul 06.

Act 7

Ringkasan & Lanjutan

Apa yang dibawa pulang, dan apa berikutnya

Ringkasan Modul 03

Bag.	#	Topik	Yang dikuasai
A	1	Pipeline pra-pemrosesan	Teks mentah jadi token bersih
A	2	POS tagging	Menandai kelas kata
A	3	Tokenisasi frasa	Entitas multi-kata tetap utuh
A	4	Stopword & lemmatisasi	Menekan kata umum, kembalikan bentuk dasar
B	5	Frekuensi kata	Menebak topik, word cloud vs bar chart
B	6	NER	Teks bebas jadi data terstruktur
C	7	Sentimen	Leksikon vs transformer (IndoBERT)
C	8	Klasifikasi Naive Bayes	Bag-of-words tanpa kebocoran data
D	9	Kata ke vektor	TF-IDF, embedding, semantic search
D	10	Tokenisasi subword	Arti "token" mulai Modul 04
NB 02		NLP di GPU	RAPIDS, nvtext, MinHash, benchmark jujur

Tiga Hal yang Dibawa Pulang

1. Pembersihan menentukan

Tanpa membuang stop-word, analisis frekuensi apa pun hanya memunculkan yang dan di.

2. Fit hanya pada train

Daftar fitur, vectorizer, scaler. Melanggarnya tidak memberi error — hanya akurasi palsu.

3. Hitungan $\neq$ makna

TF-IDF buta terhadap sinonim, embedding tidak. Perbedaan itulah yang membuat RAG mungkin.

Notebook 01: Peta Section

Isi:

- ▶ Bagian A — pembersihan teks (Section 1–4)
- ▶ Bagian B — frekuensi & NER (Section 5–6)
- ▶ Bagian C — sentimen & klasifikasi (Section 7–8)
- ▶ Bagian D — vektor & subword (Section 9–10)
- ▶ Latihan: 4 soal

Library:

- ▶ `nlp-id` — tokenizer, POS, stopword, lemmatizer
- ▶ `nltk` — korpus `movie_reviews`, Naive Bayes
- ▶ `spaCy` — NER Inggris
- ▶ `textblob` — sentimen leksikon
- ▶ `sentence-transformers` — embedding
- ▶ `transformers` — IndoBERT, tokenizer subword

Runtime: CPU cukup · **Estimasi:** $\pm$ 60 menit

Notebook 02: Peta Babak

Isi:

- ▶ Babak 1 — nol perubahan kode di GPU
- ▶ Babak 2 — nvtext pada 150 ribu paragraf Wikipedia
- ▶ Babak 3 — dedup MinHash
- ▶ Babak 4 — cuML native + pelajaran warm-up
- ▶ Babak 5 — NeMo Curator dan jembatan ke LLM

Data:

- ▶ SmSA (IndoNLU) — ulasan Indonesia berlabel sentimen
- ▶ Wikipedia Bahasa Indonesia via streaming

Runtime: wajib T4 GPU · **Estimasi:** $\pm$ 45 menit

Setup & Tools

Notebook 01 (CPU):

```
1 !pip install "nlp-id>=0.1.23,<0.2" \  
2   "transformers>=5,<6" \  
3   "sentence-transformers>=5,<6" \  
4 !python -m spacy download  
   en_core_web_sm
```

Yang mudah salah:

- ▶ nlp-id mengunci huggingface-hub 1.x, jadi transformers harus 5.x
- ▶ Colab minta **restart session** setelah instalasi — lakukan, jangan ulangi pip
- ▶ Notebook 02 butuh runtime **T4 GPU**; RAPIDS sudah terpasang

Lanjutan: Modul 04 – LLM

- ✓ **Modul 04** — Large Language Models: pipeline dari Section 7 dan "token" dari Section 10 jadi pintu masuknya
- ✓ **Modul 05** — RAG: pola embedding + semantic search dari Section 9, kali ini dengan FAISS
- ✓ **Modul 06** — Ekosistem NVIDIA: NeMo dan TensorRT-LLM, lanjutan langsung dari RAPIDS hari ini



Terima Kasih!

Pertanyaan? Diskusi?

NLP Fundamentals · Modul 03
Navasena Gen-ML Course